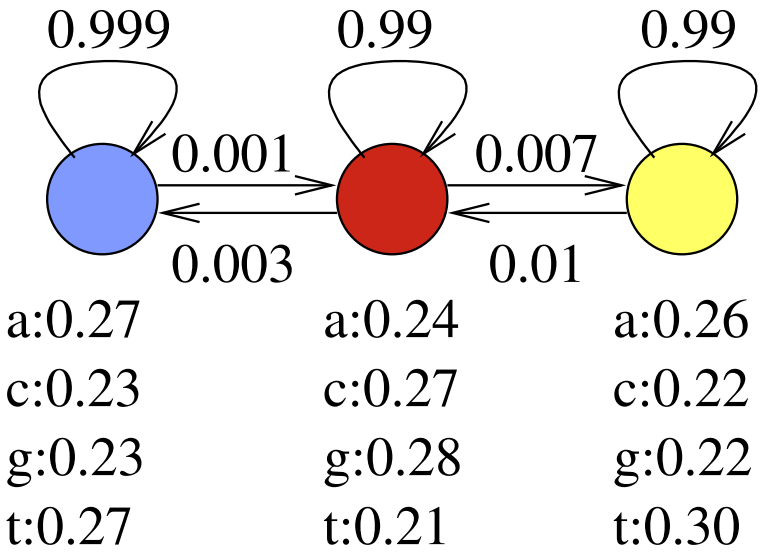
Algorithms for HMMs

(Hidden Markov Models, skrytý Markovov model)

Luca Denti

November 5, 2026



Based on lecture materials by doc. Mgr. Broňa Brejová, PhD.

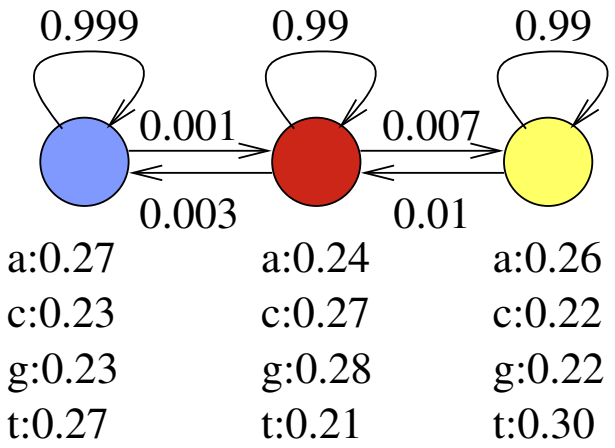
Toy example: weather

- Period of low atmospheric pressure: mostly raining
- Period of high atmospheirc pressure: mostly sunny
- Each period typically lasts several days

Exercise: Represent by an HMM

Example: genomic sequence

Task: genomic sequence G with annotation A (exonic, intronic, intergenic)



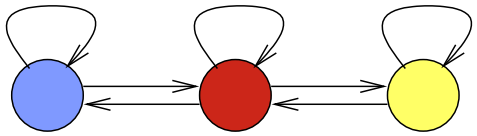
$\Pr(A, G) = \Pr(S, O)$ – probability to generate (S, O), where

- S, state path (sequence of hidden states) - $BRRR$ (annotation)
- O, observations (sequence of emissions) - $acag$ (genomic sequence)

$$\Pr(\text{acag}) = 0.27 \cdot 0.001 \cdot 0.27 \cdot 0.99 \cdot 0.24 \cdot 0.99 \cdot 0.28 = 4.8 \cdot 10^{-6}$$

$$\Pr(\text{acag}) = 0.27 \cdot 0.999 \cdot 0.23 \cdot 0.999 \cdot 0.27 \cdot 0.999 \cdot 0.23 = 0.0038$$

Parameters of HMMs (notation)



State path $S = S_1, \ldots, S_n$

Observations $O = O_1, \ldots, O_n$

Model parameters:

- Transition probability $a(u, v) = \Pr(S_{i+1} = v | S_i = u)$
- Emission probability $e(u, x) = \Pr(O_i = x | S_i = u)$
- Starting probability $\pi(u) = \Pr(S_1 = u)$

a			
	0.99	0.007	0.003
	0.01	0.99	0
	0.001	0	0.999

e	a	c	g	t
	0.24	0.27	0.28	0.21
	0.26	0.22	0.22	0.30
	0.27	0.23	0.23	0.27

The resulting probability:

$$\Pr(S, O) = \pi(S_1) e(S_1, O_1) \prod_{i=2}^{n} a(S_{i-1}, S_i) e(S_i, O_i)$$

Viterbi algorithm

For a given HMM and observations O,

find the most probable state path S^* (the one maximizing the joint probability)

$$S^* = \arg\max_S \Pr(S \mid O) = \arg\max_S \frac{\Pr(O \mid S)\Pr(S)}{\Pr(O)}$$
$$= \arg\max_S \Pr(O \mid S)\Pr(S) = \arg\max_S \Pr(S, O)$$

[Note: O is given, so Pr(O) is constant across candidate S]

Any ideas?

Recall our example:

$$\Pr(\text{acag}) = 0.27 \cdot 0.001 \cdot 0.27 \cdot 0.99 \cdot 0.24 \cdot 0.99 \cdot 0.28 = 4.8 \cdot 10^{-6}$$

$$\Pr(\text{acag}) = 0.27 \cdot 0.999 \cdot 0.23 \cdot 0.999 \cdot 0.27 \cdot 0.999 \cdot 0.23 = 0.0038$$

Viterbi algorithm

For a given HMM and observations O,

find the most probable state path S^* (the one maximizing the joint probability)

$$S^* = \arg\max_S \Pr(S \mid O) = \arg\max_S \frac{\Pr(O \mid S)\Pr(S)}{\Pr(O)}$$
$$= \arg\max_S \Pr(O \mid S)\Pr(S) = \arg\max_S \Pr(S, O)$$

[Note: O is given, so Pr(O) is constant across candidate S]

Any ideas? Brute force, $O(nm^n)$. Once again: **dynamic programming...**

Recall our example:

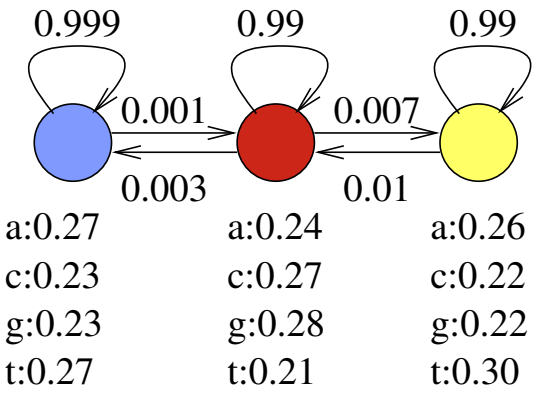
$$\Pr(\text{acag}) = 0.27 \cdot 0.001 \cdot 0.27 \cdot 0.99 \cdot 0.24 \cdot 0.99 \cdot 0.28 = 4.8 \cdot 10^{-6}$$

$$\Pr(\text{acag}) = 0.27 \cdot 0.999 \cdot 0.23 \cdot 0.999 \cdot 0.27 \cdot 0.999 \cdot 0.23 = 0.0038$$

Viterbi algorithm

Find the most probable state path $S^* = \arg\max_S \Pr(S, O)$

Subproblem $V[u, i]$: probability of the most probable state path generating $O_1 O_2 \ldots O_i$ and ending in state u



$V[u,i]$	a	c	a	g

Viterbi algorithm

Subproblem $V[u, i]$: probability of the most probable state path generating $O_1 O_2 \ldots O_i$ and ending in state u

Recurrence?

$V[u, 1] =$

$V[u, i] =$

Recall notation:

- State path $S = S_1, \ldots, S_n$, observations $O = O_1, \ldots, O_n$
- Transition probability $a(u, v) = \Pr(S_{i+1} = v | S_i = u)$
- Emission probability $e(u, x) = \Pr(O_i = x | S_i = u)$
- Starting probability $\pi(u) = \Pr(S_1 = u)$
- $\Pr(S, O) = \pi(S_1) e(S_1, O_1) \prod_{i=2}^{n} a(S_{i-1}, S_i) e(S_i, O_i)$

Viterbi algorithm

Subproblem $V[u, i]$: probability of the most probable state path generating $O_1 O_2 \ldots O_i$ and ending in state u

Recurrence:

$$V[u, 1] = \pi_u \cdot e(u, O_1)$$

$$V[u, i] = \max_w V[w, i-1] \cdot a(w, u) \cdot e(u, O_i)$$

Recall notation:

- State path $S = S_1, \ldots, S_n$, observations $O = O_1, \ldots, O_n$
- Transition probability $a(u, v) = \Pr(S_{i+1} = v | S_i = u)$
- Emission probability $e(u, x) = \Pr(O_i = x | S_i = u)$
- Starting probability $\pi(u) = \Pr(S_1 = u)$
- $\Pr(S, O) = \pi(S_1) e(S_1, O_1) \prod_{i=2}^{n} a(S_{i-1}, S_i) e(S_i, O_i)$

Viterbi algorithm (overview)

Goal: Find the most probable state path $S^* = \arg\max_S \Pr(S, O)$

Subproblem $V[u, i]$: probability of the most probable state path generating $O_1 O_2 \ldots O_i$ and ending in state u

Recurrence:

$$V[u, 1] = \pi_u \cdot e(u, O_1)$$

$$V[u, i] = \max_w V[w, i-1] \cdot a(w, u) \cdot e(u, O_i)$$

Algorithm:

Initialize $V[*, 1]$

for $i = 2 \ldots n$ (n=length of O)

    for $u = 1 \ldots m$ (m =number of states)

        compute $V[u, i]$, keep best w in $B[u, i]$

Maximum $V[u, n]$ over all u is $\max_S \Pr(S, O)$

Retrieve the full path using matrix B

Running time: dynamic programming in $O(nm^2)$ time

Second problem: overall probability of O

Viterbi computes $\arg\max_S \Pr(S, O)$

Now we want $\Pr(O) = \sum_S \Pr(S, O)$

Useful e.g. to compare different models: which is more likely to produce O?

Any ideas?

Recall our example:

$$\Pr(\text{acag}) = 0.27 \cdot 0.001 \cdot 0.27 \cdot 0.99 \cdot 0.24 \cdot 0.99 \cdot 0.28 = 4.8 \cdot 10^{-6}$$

$$\Pr(\text{acag}) = 0.27 \cdot 0.999 \cdot 0.23 \cdot 0.999 \cdot 0.27 \cdot 0.999 \cdot 0.23 = 0.0038$$

Forward algorithm (dopredný algoritmus)

Computes overall probability that the model emits O, $\Pr(O) = \sum_S \Pr(S, O)$

Subproblem $F[u, i]$: probability that in i steps we generate $O_1, O_2, \ldots O_i$ and end in state u.

$$F[u, i] = \Pr(S_i = u \wedge O_1, O_2, \ldots, O_i) =$$

$$\sum_{S_1, \ldots, S_{i-1}, S_i = u} \Pr(S_1, S_2, ..., S_i \wedge O_1, O_2, ..., O_i)$$

Recurrence?

$$F[u, 1] =$$

$$F[u, i] =$$

Recall Viterbi recurrence:

$$V[u, 1] = \pi_u \cdot e(u, O_1)$$

$$V[u, i] = \max_w V[w, i-1] \cdot a(w, u) \cdot e(u, O_i)$$

Forward algorithm

Computes overall probability that the model emits O, $\Pr(O) = \sum_S \Pr(S, O)$

Subproblem $F[u, i]$: probability that in i steps we generate $O_1, O_2, \ldots O_i$ and end in state u.

$$F[u, i] = \Pr(S_i = u \wedge O_1, O_2, \ldots, O_i) =$$

$$\sum_{S_1, \ldots, S_{i-1}, S_i = u} \Pr(S_1, S_2, ..., S_i \wedge O_1, O_2, ..., O_i)$$

Recurrence:

$$F[u, 1] = \pi_u \cdot e(u, O_1)$$

$$F[u, i] = \sum_w F[w, i-1] \cdot a(w, u) \cdot e(u, O_i)$$

Recall Viterbi recurrence:

$$V[u, 1] = \pi_u \cdot e(u, O_1)$$

$$V[u, i] = \max_w V[w, i-1] \cdot a(w, u) \cdot e(u, O_i)$$

Forward algorithm

Computes overall probability that the model emits O, regardless of state path.

$$\Pr(O) = \sum_A \Pr(A, S)$$

Subproblem $F[u, i]$: probability that in i steps we generate $O_1, O_2, \ldots O_i$ and end in state u.

Recurrence:

$$F[u, 1] = \pi_u \cdot e(u, O_1)$$

$$F[u, i] = \sum_w F[w, i-1] \cdot a(w, u) \cdot e(u, O_i)$$

Result? $\Pr(S) =$

Running time?

Forward algorithm

Computes overall probability that the model emits O, regardless of state path.

$$\Pr(O) = \sum_A \Pr(A, S)$$

Subproblem $F[u, i]$: probability that in i steps we generate $O_1, O_2, \ldots O_i$ and end in state u.

Recurrence:

$$F[u, 1] = \pi_u \cdot e(u, O_1)$$

$$F[u, i] = \sum_w F[w, i-1] \cdot a(w, u) \cdot e(u, O_i)$$

Result: $\Pr(S) = \sum_u F[u, n]$

Running time: $O(nm^2)$

Third problem: probability that O_i was emitted in state u

$$\Pr(S_i = u \,|\, O) = \frac{\Pr(S_i = u, O)}{\Pr(O)}$$

where

$$\Pr(S_i = u, O) = \sum_{S : S_i = u} \Pr(S, O)$$

Compute this by a combination of forward and backward algorithms

- $F[u, i]$: probability that in i steps we generate $O_1, O_2, \ldots O_i$ and end in state u.
- $B[u, i]$: probability that if we start at u at position i, we will generate $O_{i+1} \ldots, O_n$ in the next steps

$$\Pr(S_i = u, O) = F[u, i] \cdot B[u, i]$$

Backward algorithm (spätný algoritmus)

Forward algorithm $F[u, i]$: probability that in i steps we generate $O_1, O_2, \ldots O_i$ and end in state u.

$$F[u, 1] = \pi_u \cdot e(u, O_1)$$

$$F[u, i] = \sum_w F[w, i-1] \cdot a(w, u) \cdot e(u, O_i)$$

Backward algorithm $B[u, i]$: probability that if we start at u at position i, we will generate $O_{i+1} \ldots, O_n$ in the next steps

How to compute $B[u, i]$?

Backward algorithm (spätný algoritmus)

Forward algorithm $F[u,i]$: probability that in i steps we generate $O_1, O_2, \ldots O_i$ and end in state u.

$$F[u,1] = \pi_u \cdot e(u, O_1)$$

$$F[u,i] = \sum_w F[w, i-1] \cdot a(w,u) \cdot e(u, O_i)$$

Backward algorithm $B[u,i]$: probability that if we start at u at position i, we will generate $O_{i+1} \ldots, O_n$ in the next steps

$$B[u,n] = 1$$

$$B[u,i] = \sum_w B[w, i+1] \cdot a(u,w) \cdot e(w, O_{i+1})$$

Use cases

Using F and B we can compute **matrix of posterior probabilities**,

$$\Pr(S_i = u \mid O) \textit{ for each } u \textit{ and } i$$

in $O(nm^2)$ overall time (+ normalize by $P(O)$, as sum of column).

Use 1: posterior decoding, select S such that $S_i = \max_i \Pr(S_i = u \mid O)$

Advantage: this takes into account suboptimal state paths

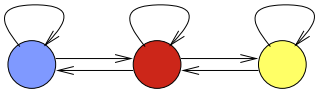
Disadvantage: $\Pr(S \mid O)$ can be very low (unlikely path) or zero (forbidden transitions)

Use 2: use posterior probabilities to assign confidence to parts of prediction from Viterbi

Recall: Finding genes with HMMs

Task: genomic sequence G with annotation A (exonic, intronic, intergenic)

$$\Pr(A, G) = \Pr(S, O)$$ – probability to generate (S, O)

1. **Determine states and transitions of the model:** by hand based on your knowledge about the gene structure (biology) 
2. **Parameter training:** emission and transition probabilities are determined based on the real sequences with known genes (**training set**)
3. **Use:** for a new sequence G, find the most probable annotation

$$A^* = \arg\max_A \Pr(A|G)$$

using Viterbi algorithm in $O(nm^2)$ (dynamic programming)

Parameter training

Typically:

- States and allowed transitions manually
- Probabilities (transition, emission, starting) automatically from training data

More complex models with more parameters need more training data

Problem: overfitting

- model fits training data very well but behaves poorly on unseen examples)
- to test acurracy of the model use a separate testing set not used for training

HMM parameter (probabilities) training from annotated sequences

Input: state diagram of the model and a training set of observations and states

$$(O^{(1)}, S^{(1)}), (O^{(2)}, S^{(2)}), \ldots$$

Goal: choose parameters that maximize their likelihood in the model

$$\arg\max_{a,e,\pi} \prod_i \Pr(O^{(i)}, S^{(i)} | a, e, \pi)$$

This is achieved by using observed frequencies

For example $a(u, v)$:

find all occurrences of state u and find out how often it is followed by v

HMM parameter (probabilities) training from unannotated sequences

Input: state diagram of the model and a training set of observations

$$O^{(1)}, O^{(2)}, \ldots \text{ (states } S^{(i)} \text{ are unknown)}$$

Goal: choose parameters maximalizing their likelihood in the model

$$\arg\max_{a,e,\pi} \prod_i \Pr(O^{(i)}|a, e, \pi)$$

Baum-Welch algorithm (version of expectation maximization, EM)

- Iterative heuristic algorithm improving parameters until convergence
- Each iteration forward and backward algorithms